

Constructors, Destructors & Object Lifecycle

1. Constructors — initializing an object correctly, automatically

A **constructor** is a special method, automatically called when an object is created, whose job is to put the object into a valid initial state.

- Same name as the class (C++/Java), no return type at all (not even `void`).
- Called **exactly once**, automatically, at creation — never called manually like a regular method.

Default constructor

Takes no arguments; if you don't write **any** constructor, the compiler auto-generates a trivial one for you (in C++, it default-initializes members; in Java, it zero/null-initializes fields).

```
// C++
class Cat {
public:
    string name, colour, toy;
    Cat() { name = "Unknown"; colour = "Unknown"; toy = "Unknown"; }
};
```

```
// Java
class Cat {
    String name, colour, toy;
    Cat() { name = "Unknown"; colour = "Unknown"; toy = "Unknown"; }
}
```

Important trap: the moment you define **any** constructor yourself (e.g., a parameterized one), the compiler **stops** auto-generating the default (no-argument) constructor — if you still need a no-argument way to create the object, you must write it explicitly.

Parameterized constructor & constructor overloading

Multiple constructors, differing in parameter list — this is just **function overloading** (see the Polymorphism topic) applied specifically to constructors, letting an object be created in more than one way.

```
class Number {
public:
    int n;
    Number(int v) { n = v; }           // parameterized
    Number() { n = 0; }                // overloaded default
};
```

Constructor initialization lists (C++-specific nuance)

```
class EquilateralTriangle {
    int side, area, circumference;
public:
    EquilateralTriangle(int s) : side(s) {    // initialization list
        area = 1.732 * side * side / 2;
        circumference = 3 * side;
    }
};
```

Why prefer an initialization list over assigning inside the constructor body? Members listed here are **initialized directly** as they're constructed, rather than first being default-constructed and then reassigned inside the body — for simple types the difference is negligible, but for members that are `const`, references, or objects without a default constructor, an initialization list is **mandatory** — you cannot assign to a `const` member or a reference after construction, only initialize it at the moment it comes into existence.

2. The Copy Constructor & the Shallow vs. Deep Copy Problem

A **copy constructor** creates a new object as a copy of an existing one of the same class.

```
class Number {
public:
    int n;
    Number(int v) { n = v; }
};

int main() {
    Number n1(100);
    Number n2 = n1;    // the DEFAULT (compiler-generated) copy constructor is called here
    cout << n1.n << " " << n2.n;    // 100 100
}
```

If you don't write one yourself, the compiler generates a **default copy constructor** that copies each member's value directly — this works fine for simple value members like `int`, but becomes a serious problem the moment a member is a **pointer**.

The shallow copy problem

```
class Buffer {
    int* data;
public:
    Buffer(int size) { data = new int[size]; }
    // no custom copy constructor written — compiler generates a SHALLOW copy
};

Buffer a(10);
Buffer b = a;    // b.data now points to the SAME memory as a.data!
```

- **Shallow copy** — copies the **pointer value itself** (the address), not what it points to. Now `a.data` and `b.data` point to the **exact same** heap memory.

- **Consequences:** modifying data through `b` is visible through `a` too (probably not intended); and worse, when both objects are destroyed, **both destructors** will try to `delete[] data` on the **same memory** — a **double-free**, undefined behavior that commonly crashes the program.

The fix: a custom (deep) copy constructor

```
class Buffer {
    int* data;
    int size;
public:
    Buffer(int s) : size(s) { data = new int[size]; }
    Buffer(const Buffer& other) : size(other.size) { // DEEP copy constructor
        data = new int[size]; // allocate SEPARATE memory
        for (int i = 0; i < size; i++) data[i] = other.data[i]; // copy the actual VALUES
    }
    ~Buffer() { delete[] data; }
};
```

- **Deep copy** — allocates **new, independent** memory and copies the actual pointed-to *values* — the two objects now have completely independent data; modifying one never affects the other, and each object's destructor safely frees its own memory.

Interview soundbite: "Whenever a class manages a resource through a raw pointer (heap memory, a file handle, a socket), the compiler-generated default copy constructor is almost always wrong — it copies the *address*, not the *resource*. This is exactly the motivation behind the 'Rule of Three' in C++: if a class needs a custom destructor, it almost certainly also needs a custom copy constructor and copy assignment operator, because all three are really about the same underlying question — who owns this resource, and what happens when an object holding it is copied or destroyed?"

Java note: Java has no raw pointers and no manual copy constructors as a language feature — assignment (`Cat b = a;`) always copies a **reference**, and Java's garbage collector (not the developer) handles freeing memory once nothing references it anymore, sidestepping the double-free class of bug entirely. To get an actual independent copy in Java, you'd implement `Cloneable` / a custom `copy()` method — and the exact same shallow-vs-deep distinction still applies there (a naive `clone()` copies references to any mutable fields, not their contents).

3. The `this` Keyword/Pointer

`this` refers to the **current object** — the specific instance a method is currently executing on.

The classic use case: disambiguating a parameter from a field of the same name.

```
class Car {
    double price;
    void applyDiscount(double price) { // parameter 'price' HIDES the instance field 'price'
        this.price = this.price - price; // this.price = the field; price (bare) = the parameter
    }
}
```

```
class Car {
    double price;
public:
    void applyDiscount(double price) {
        this->price = this->price - price; // C++ uses -> since `this` is a pointer
    }
};
```

Key difference: in Java, `this` is effectively a reference; in C++, `this` is literally a **pointer** to the current object — which is exactly why C++ uses `this->field` (pointer member access) while Java uses `this.field` (reference/object member access).

Other uses of `this`: returning `*this` (C++) or `this` (Java) from a method to enable **method chaining** (see the Operator Overloading topic for the C++ mechanics), and passing the current object to another function/constructor.

4. Destructors

A **destructor** is automatically called when an object's lifetime ends, responsible for releasing any resources the object acquired (freeing heap memory, closing files, releasing locks).

```
class Buffer {
    int* data;
public:
    Buffer(int size) { data = new int[size]; }
    ~Buffer() { delete[] data; } // destructor: same name as class, prefixed with ~, no return type, no parameters
};
```

- Called **automatically** — when a stack-allocated object goes out of scope, or explicitly via `delete` for a heap-allocated object.
- Exactly **one** destructor per class — it takes **no parameters** and cannot be overloaded (there's nothing to overload it *with*, since it's never called with arguments).

Virtual destructors — a genuinely important, frequently-tested nuance

```
class Animal {
public:
    virtual ~Animal() { cout << "Animal destroyed"; } // VIRTUAL destructor
};
class Dog : public Animal {
    int* extraData;
public:
    Dog() { extraData = new int[100]; }
    ~Dog() { delete[] extraData; cout << "Dog destroyed"; }
};

Animal* a = new Dog();
delete a; // which destructor(s) actually run?
```

- If `~Animal()` is **virtual**: `delete a` correctly calls `~Dog()` **first** (which cleans up `extraData`), **then** `~Animal()` — full, correct cleanup.
- If `~Animal()` is **NOT virtual**: `delete a` calls **only** `~Animal()` — `~Dog()` is **never called at all**, `extraData` is **never freed** — a memory leak, and any Dog-specific cleanup silently never happens.

The rule to remember: any class intended to be used polymorphically (through a base-class pointer, with derived classes) must declare its destructor `virtual` — this single omission is one of the most common, genuinely dangerous C++ bugs in interview settings, precisely because the code *compiles fine* and only fails silently at runtime via a leak.

Java note: Java has no destructors in this sense at all — the garbage collector reclaims memory automatically once an object is unreachable, and *when* that happens is not deterministic or under direct program control. For deterministic cleanup of non-memory resources (files, sockets, DB connections), Java uses `try-with-resources` and the `AutoCloseable` interface's `close()` method instead — a fundamentally different mechanism, precisely because "when does this object die" isn't something Java code controls directly the way C++ does.

5. Object Lifecycle: Stack vs. Heap (C++-specific, but conceptually relevant everywhere)

```
{
    Cat c1; // STACK — automatic storage; destructor called automatically at end of scope
    Cat* c2 = new Cat(); // HEAP — persists until explicitly deleted
} // c1's destructor runs HERE automatically
// c2 still exists — its memory is LEAKED unless `delete c2;` was called before this point
```

	Stack allocation	Heap allocation
Syntax	<code>Cat c1;</code>	<code>Cat* c2 = new Cat();</code>
Lifetime	Automatic — ends when scope ends	Manual — until explicit <code>delete</code>
Destructor call	Automatic	Only on explicit <code>delete</code>
Risk	None (as long as scope is well-defined)	Memory leak if <code>delete</code> is forgotten; dangling pointer if used after <code>delete</code>

Java note: all objects in Java are allocated on the heap (there's no direct stack-object equivalent for user-defined types) — memory management risk shifts entirely from "did I remember to free this" (C++'s risk) to "am I unintentionally keeping a reference alive longer than needed" (Java's risk — a *logical* memory leak, where garbage collection can't reclaim something because some reachable reference still points to it, even though the program logically no longer needs it).

Interview Questions With Answers

Q1. Why does defining a parameterized constructor yourself remove the compiler's automatically-generated default constructor?

Answer: The compiler only auto-generates a default (no-argument) constructor when the class defines **no** constructors at all — this is meant purely as a convenience for classes that don't need any custom initialization logic. The instant you write any constructor yourself, the compiler assumes you're taking full responsibility for how the class is constructed, and stops providing the implicit default — if a no-argument way to construct the object is still needed, it must now be written explicitly, since the compiler no longer infers that one is wanted.

Q2. Explain, with an example, exactly what goes wrong with a shallow copy of an object holding a raw pointer to heap memory.

Answer: A shallow copy (the compiler's default copy constructor, if none is written) copies a pointer member's *value* — the memory address it holds — not the data it points to. If class `Buffer` has an `int* data` member, copying a `Buffer` object this way results in two separate `Buffer` objects whose `data` pointers point to the *exact same* underlying heap memory. This causes two problems: modifying the data through one object is silently visible through the other (likely unintended), and when both objects are eventually destroyed, both destructors will attempt to `delete` the *same* memory — a double-free, which is undefined behavior and commonly crashes the program or corrupts the heap.

Q3. How does a custom (deep) copy constructor fix the shallow copy problem, and what must it specifically do differently?

Answer: A deep copy constructor must allocate its **own, separate** block of memory (rather than copying the source pointer's address), and then copy the actual *values* from the source's memory into this newly allocated memory, element by element. This ensures the two objects' pointers point to entirely independent memory regions — modifying one object's data never affects the other, and each object's destructor safely frees only its own independently-allocated memory, eliminating the double-free risk entirely.

Q4. Why is `this` a pointer in C++ but effectively a reference in Java, and how does this show up syntactically?

Answer: This reflects each language's underlying object model: C++ objects can be manipulated directly via pointers as a first-class language feature, and `this` is defined as a pointer to the current object, consistent with how C++ generally handles indirect access — which is why C++ code accesses members through `this` using the pointer-member-access operator `this->field`. Java has no raw pointers as a language-level concept at all; `this` behaves like an object reference, and Java uses ordinary dot notation `this.field` for member access, the same syntax used for any object reference in Java.

Q5. Walk through exactly what happens (and what goes wrong) when a Dog object is deleted through a non-virtual Animal base class pointer.

Answer: When `delete` is called on a pointer whose static (compile-time) type is `Animal`, and `~Animal()` is **not** declared virtual, the compiler resolves which destructor to call based on the pointer's static type alone — it calls only `~Animal()`, completely ignoring the fact that the object's actual runtime type is `Dog`. This means `~Dog()` is never invoked at all — any cleanup logic specific to Dog (like freeing a Dog-specific heap allocation) is silently skipped, causing a memory leak (or worse, resource leaks of any kind Dog was responsible for cleaning up) that won't produce any compiler warning or runtime crash — it just quietly leaks.

Q6. Why must the destructor of any class intended to be used polymorphically (through a base class pointer) be declared virtual?

Answer: Polymorphic use means code holds a pointer/reference of the base class's type but the actual object could be any derived class — and when that object is destroyed via `delete` (or falls out of scope through a base-class reference mechanism), the correct, most-derived destructor needs to run to properly release whatever resources that specific derived class acquired, followed by the base class's own destructor. Declaring the destructor virtual makes the compiler resolve *which* destructor to call based on the object's actual runtime type (via the virtual dispatch mechanism, covered in the Polymorphism topic) rather than the pointer's static type — without this, only the base class's destructor would ever run, regardless of the object's true type, silently skipping derived-class cleanup.

Q7. Why doesn't Java have destructors in the same sense C++ does, and what mechanism does Java use instead for deterministic resource cleanup?

Answer: Java's garbage collector reclaims an object's memory automatically once it determines the object is no longer reachable from any live reference — but exactly *when* this happens is not deterministic and isn't directly controlled by the program, which makes a C++-style destructor (guaranteed to run at a specific, predictable point — end of scope, or explicit delete) fundamentally incompatible with how Java manages memory. For resources that genuinely need deterministic, guaranteed cleanup at a known point (files, sockets, database connections — not just memory, which garbage collection handles fine on its own), Java instead provides the `AutoCloseable` interface and `try-with-resources` syntax, which guarantees a `close()` method is called at a specific, predictable point (the end of the try block), independent of whenever garbage collection eventually happens to reclaim the object's memory.

Q8. Scenario: A C++ class `Matrix` stores its elements in a dynamically-allocated `double*` `data` array. A developer writes the class with a custom constructor and destructor, but doesn't write a copy constructor or copy assignment operator, reasoning "the compiler will generate sensible defaults." Later, a function that takes a `Matrix` by value (not by reference) is called, and the program crashes with heap corruption shortly after. Diagnose the bug.

Answer: Passing a `Matrix` by value invokes the copy constructor to create the function's local parameter copy — since none was written, the compiler-generated default copy constructor performs a shallow copy, meaning the function's local `Matrix` copy's `data` pointer points to the *exact same* heap memory as the original `Matrix` passed in. When the function returns, its local copy goes out of scope and its destructor runs, `delete`-ing that shared memory. The *original* `Matrix` object (still alive in the caller) now holds a dangling pointer to already-freed memory — any subsequent use of it is undefined behavior, and when the original `Matrix`'s own destructor eventually runs, it attempts to delete the *same* memory a second time (a double-free), which is exactly the kind of heap corruption described. The fix is exactly the "Rule of Three" reasoning from earlier: since `Matrix` manages a raw-pointer resource and needed a custom destructor, it also needed a custom (deep) copy constructor and copy assignment operator — writing the destructor alone was the red flag that the other two were also required.